

UNIT I COMPONENTS OF AN OS

Computer System -Elements and organization; Operating System Overview-Objectives and Functions - Evolution of Operating System; Operating System Structures – Operating System Services - User Operating System Interface - System Calls – System Programs.

COMPUTER SYSTEM

A computer system is a collection of hardware and software components designed to provide an effective tool for computation.

Hardware generally refers to the electrical, mechanical and electronic parts that make up the computer (i.e., internal architecture of the computer (or) physical computing equipment). However the hardware is sophisticated, it cannot function properly without a proper driver which can drive it and bring it to the best advantage. For example, a car, even though sophisticated in its features, it cannot function independently without being properly driven by an efficient driver.

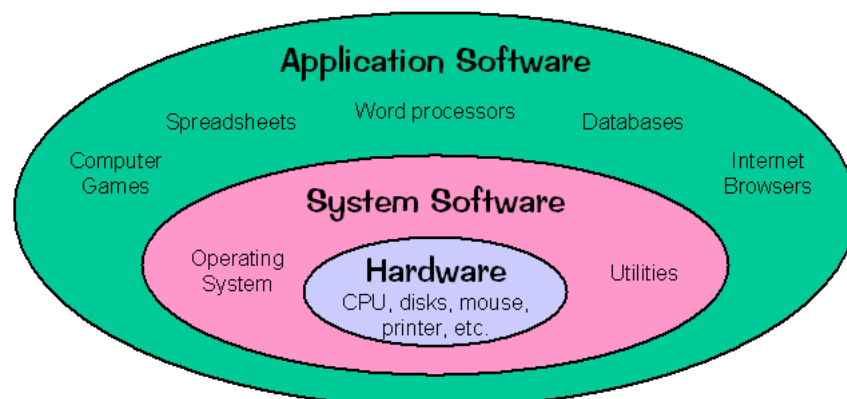
Similarly the hardware though technologically innovative, and which presents enhanced features, which needs set of programs to bring it to operation and to the best advantage. So, the driver that drives the hardware is software.

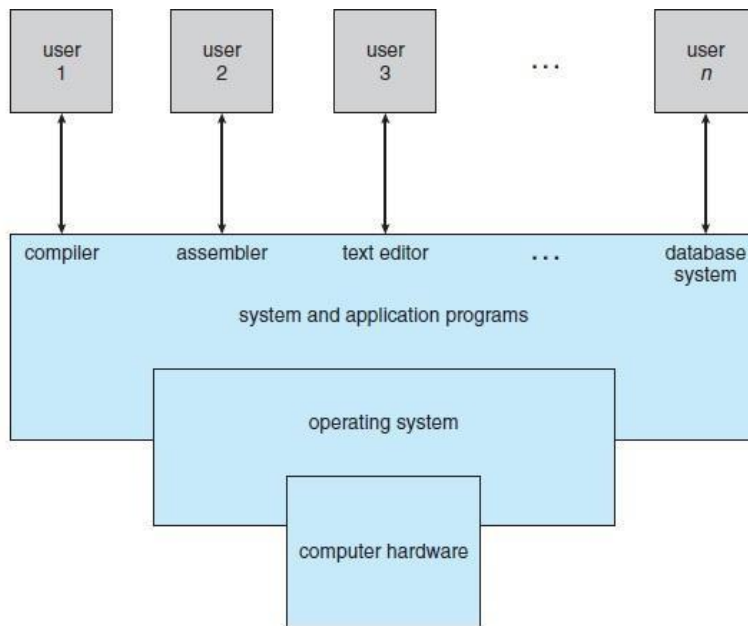
Software refers to the set of programs written to provide services to the system. It gives life and meaning to the hardware and bring it to the operational level, which otherwise is a useless piece of metal. Software is basically of two types:

1. Application software
2. System software

Application Software: Set of programs written for a specific area of application. For example, word processors, spreadsheets and data base management systems, etc.

System Software: Set of programs written from the point of view of the machine i.e., for the sake of the system. System software provides environment for execution of application software. One cannot aim to develop or write application software, without the presence and aid of system software.





Abstract view of the components of a computer system.

used to solve users' computing problems.

The operating system controls the hardware and coordinates its use among the various application programs for the various users.

We can also view a computer system as consisting of hardware, software, and data.

Two Views of Operating System

1. User's View
2. System View

User View:

The user's view of the computer varies according to the interface being used. Most computer users sit in front of a PC, consisting of a monitor, keyboard, mouse, and system unit. Such a system is designed for one user to monopolize its resources. The goal is to maximize the work (or play) that the user is performing. In this case, the operating system is designed mostly for **ease of use**, with some attention paid to performance and none paid to **resource utilization**—how various hardware and software resources are shared. Performance is, of course, important to the user; but such systems are optimized for the single-user experience rather than the requirements of multiple users.

In other cases, a user sits at a terminal connected to a **mainframe** or a **minicomputer**. Other users are accessing the same computer through other terminals. These users share resources and may exchange information. The operating system in such cases is designed to maximize resource utilization to assure

A computer system can be divided roughly into **four components**: the **hardware**, the **operating system**, the **application programs**, and the **users**.

The **hardware**—the **central processing unit (CPU)**, the **memory**, and the **input/output (I/O) devices**—provides the basic computing resources for the system.

The **application programs**—such as word processors, spreadsheets, compilers, and Web browsers—define the ways in which these resources are

that all available CPU time, memory, and I/O are used efficiently and that no individual user takes more than her fair share.

In other cases, users sit at **workstations** connected to networks of other workstations and **servers**.

Recently, many varieties of mobile computers, such as smart phones and tablets, have come into fashion. The user interface for mobile computers generally features a **touch screen**, where the user interacts with the system by pressing and swiping fingers across the screen rather than using a physical keyboard and mouse.

System View:

From the computer's point of view, the operating system is the program most intimately involved with the hardware. In this context, we can view an operating system as a **resource allocator**.

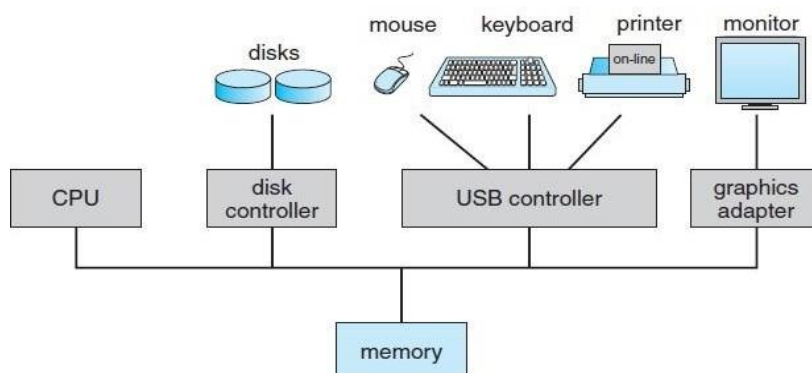
A computer system has many resources that may be required to solve a problem: CPU time, memory space, file-storage space, I/O devices, and so on. The operating system acts as the manager of these resources.

A slightly different view of an operating system emphasizes the need to control the various I/O devices and user programs. An operating system is a control program. A **control program** manages the execution of user programs to prevent errors and improper use of the computer. It is especially concerned with the operation and control of I/O devices.

COMPUTER SYSTEM ELEMENTS & ORGANIZATION

Before we can explore the details of how computer systems operate, we need general knowledge of the structure of a computer system. In this section, we look at several parts of this structure. The section is mostly concerned with computer-system organization, so you can skim or skip it if you already understand the concepts.

Computer-System Operation:



A modern computer system.

A modern general-purpose computer system consists of one or more CPUs and a number of device controllers connected through a common bus that provides access to shared memory (Figure). Each device controller is in charge of a specific type of device (for

example, disk drives, audio devices, or video displays). The CPU and the device controllers can execute in parallel, competing for memory cycles. To ensure orderly access to the shared memory, a memory controller synchronizes access to the memory.

For a computer to start running—for instance, when it is powered up or rebooted—it needs to have an initial program to run. This initial program, or **bootstrap program**, tends to be simple.

Typically, it is stored within the computer hardware in read-only memory (**ROM**) or electrically erasable programmable read-only memory (**EEPROM**), known by the general term **firmware**.

It initializes all aspects of the system, from CPU registers to device controllers to memory contents. The bootstrap program must know how to load the operating system and how to start executing that system. To accomplish this goal, the bootstrap program must locate the operating-system kernel and load it into memory.

Once the kernel is loaded and executing, it can start providing services to the system and its users. Some services are provided outside of the kernel, by system programs that are loaded into memory at boot time to become **system processes**, or **system daemons** that run the entire time the kernel is running. On UNIX, the first system process is “init,” and it starts many other daemons. Once this phase is complete, the system is fully booted, and the system waits for some event to occur.

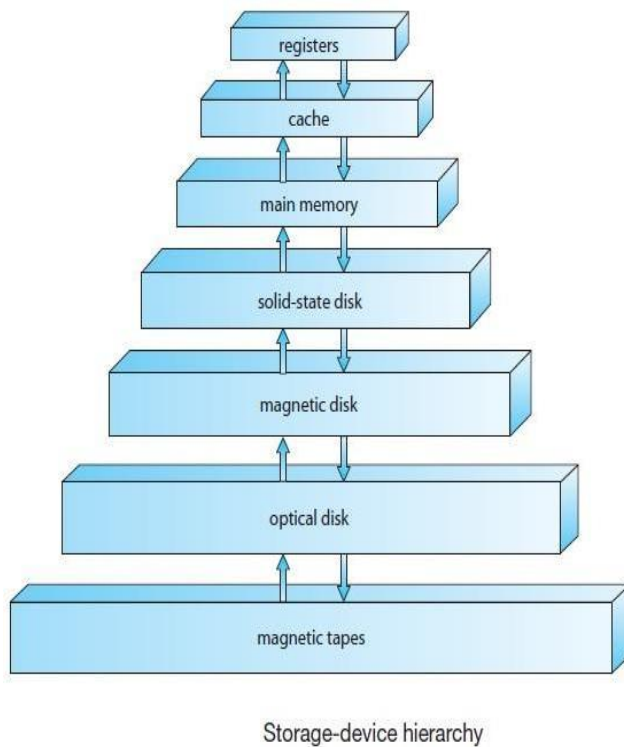
The occurrence of an event is usually signaled by an **interrupt** from either the hardware or the software. Hardware may trigger an interrupt at any time by sending a signal to the CPU, usually by way of the system bus. Software may trigger an interrupt by executing a special operation called a **system call** (also called a **monitor call**).

Storage Structure:

The CPU can load instructions only from memory, so any programs to run must be stored there. General-purpose computers run most of their programs from rewritable memory, called main memory (also called **random-access memory**, or **RAM**). Main memory commonly is implemented in a semiconductor technology called **dynamic random-access memory (DRAM)**. Ideally, we want the programs and data to reside in main memory permanently.

This arrangement usually is not possible for the following two reasons:

1. Main memory is usually too small to store all needed programs and data permanently.
2. Main memory is a **volatile** storage device that loses its contents when power is turned off or otherwise lost.



Thus, most computer systems provide **secondary storage** as an extension of main memory. The main requirement for secondary storage is that it be able to hold large quantities of data permanently. The most common secondary-storage device is a **magnetic disk**, which provides storage for both programs and data.

The wide variety of storage systems can be organized in a hierarchy (Figure) according to speed and cost. The higher levels are expensive, but they are fast. As we move down the hierarchy, the cost per bit generally decreases, whereas the access time generally increases.

Storage Definition and notation

- The basic unit of computer storage is the **bit**. A bit can contain one of two values, 0 and 1.
- All other storage in a computer is based on collections of bits.
- Given enough bits, it is amazing how many things a computer can represent: numbers, letters, images, movies, sounds, documents, and programs, to name a few.
- A **byte** is 8 bits, and on most computers it is the smallest convenient chunk of storage.
- For example, most computers don't have an instruction to move a bit but do have one to move a byte. A less common term is **word**, which is a given computer architecture's native unit of data. A word is made up of one or more bytes.
- For example, a computer that has 64-bit registers and 64-bit memory addressing typically has 64-bit (8-byte) words.
- A computer executes many operations in its native word size rather than a byte at a time.
- Computer storage, along with most computer throughput, is generally measured and manipulated in bytes and collections of bytes.

a **kilobyte**, or **KB**, is 1,024 bytes;

a **megabyte**, or **MB**, is 1,024 bytes;

a **gigabyte**, or **GB**, is 1,024 bytes;

a **terabyte**, or **TB**, is 1,024 bytes;

and a **petabyte**, or **PB**, is 1,024 bytes.

Computer manufacturers often round off these numbers and say that a megabyte is 1 million bytes and a gigabyte is 1 billion bytes. Networking measurements are an exception to this general rule; they are given in bits (because networks move data a bit at a time).

I/O Structure:

Storage is only one of many types of I/O devices within a computer. A large portion of operating system code is dedicated to managing I/O, both because of its importance to the reliability and performance of a system and because of the varying nature of the devices. Next, we provide an overview of I/O.

A general-purpose computer system consists of CPUs and multiple device controllers that are connected through a common bus. Each device controller is in charge of a specific type of device. Depending on the controller, more than one device may be attached. For instance, seven or more devices can be attached to the **small computer-systems interface (SCSI)** controller.

A device controller maintains some local buffer storage and a set of special-purpose registers. The device controller is responsible for moving the data between the peripheral devices that it controls and its local buffer storage. Typically, operating systems have a **device driver** for each device controller. This device driver understands the device controller and provides the rest of the operating system with a uniform interface to the device.

This form of interrupt-driven I/O is fine for moving small amounts of data but can produce high overhead when used for bulk data movement such as disk I/O. To solve this problem, **direct memory access (DMA)** is used.

Computer-System Organization

A computer system can be organized in a number of different ways, which we can categorize roughly according to the number of general-purpose processors used.

Single-Processor Systems

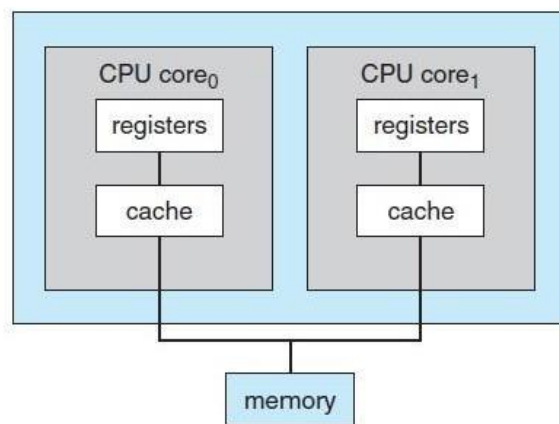
Until recently, most computer systems used a single processor. On a single processor system, there is one main CPU capable of executing a general-purpose instruction set, including instructions from user processes. Almost all single processor systems have other special-purpose processors as well. They may come in the form of device-specific processors, such as disk, keyboard, and graphics controllers; or, on mainframes, they may come in the form of more general-purpose processors, such as I/O processors that move data rapidly among the components of the system.

All of these special-purpose processors run a limited instruction set and do not run user processes. Sometimes, they are managed by the operating system, in that the operating system sends them information about their next task and monitors their status.

For example, a disk-controller microprocessor receives a sequence of requests from the main CPU and implements its own disk queue and scheduling algorithm. This arrangement relieves the main CPU of the overhead of disk scheduling. PCs contain a microprocessor in the keyboard to convert the keystrokes into codes to be sent to the CPU.

In other systems or circumstances, special-purpose processors are low-level components built into the hardware. The operating system cannot communicate with these processors; they do their jobs autonomously. The use of special-purpose microprocessors is common and does not turn a single-processor system into a multiprocessor. If there is only one general-purpose CPU, then the system is a single-processor system.

Multiprocessor Systems



A dual-core design with two cores placed on the same chip.

Within the past several years, **multiprocessor systems** (also known as **parallel systems** or **multicore systems**) have begun to dominate the landscape of computing. Such systems have two or more processors in close communication, sharing the computer bus and sometimes the clock, memory, and peripheral devices. Multiprocessor systems

first appeared prominently in servers and have since migrated to desktop and laptop systems. Recently, multiple processors have appeared on mobile devices such as smartphones and tablet computers.

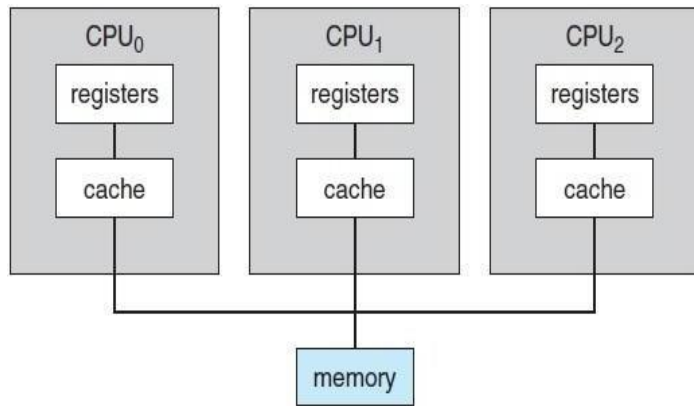
Multiprocessor systems have three main advantages:

1. **Increased throughput.** By increasing the number of processors, we expect to get more work done in less time. The speed-up ratio with N processors is not N , however; rather, it is less than N . When multiple processors cooperate on a task, a certain amount of overhead is incurred in keeping all the parts working correctly. This overhead, plus contention for shared resources, lowers the expected gain from additional processors. Similarly, N programmers working closely together do not produce N times the amount of work a single programmer would produce.

2. **Economy of scale.** Multiprocessor systems can cost less than equivalent multiple single-processor systems, because they can share peripherals, mass storage, and power supplies. If several programs operate on the same set of data, it is cheaper to store those data on one disk and to have all the processors share them than to have many computers with local disks and many copies of the data.

3. Increased reliability. If functions can be distributed properly among several processors, then the failure of one processor will not halt the system, only slow it down. If we have ten processors and one fails, then each of the remaining nine processors can pick up a share of the work of the failed processor. Thus, the entire system runs only 10 percent slower, rather than failing altogether.

The multiple-processor systems in use today are of two types.



Symmetric multiprocessing architecture.

Some systems use **asymmetric multiprocessing**, in which each processor is assigned a specific task.

A **boss** processor controls the system; the other processors either look to the boss for instruction or have predefined tasks. This scheme defines a boss-worker relationship. The boss processor schedules and allocates work to the worker processors.

The most common systems use **symmetric multiprocessing (SMP)**, in which each processor performs all tasks within the operating system. SMP means that all processors are peers; no boss-worker relationship exists between processors.

Clustered Systems:

Like parallel systems, clustered systems gather together multiple CPUs to accomplish computational work. Clustered systems differ from parallel systems, however, in that they are composed of two or more individual systems coupled together. The definition of the term clustered is **not concrete**; the general accepted definition is that clustered computers share storage and is closely linked via LAN networking. Clustering is usually performed to provide **high availability**.

A layer of cluster software runs on the cluster nodes. Each node can monitor one or more of the others. If the monitored machine fails, the monitoring machine can take ownership of its storage, and restart the application(s) that were running on the failed machine. The failed machine can remain down, but the users and clients of the application would only see a brief interruption of service.

- **Asymmetric Clustering** - In this, one machine is in hot standby mode while the other is running the applications. The hot standby host (machine) does nothing but monitor the active server. If that server fails, the hot standby host becomes the active server.
- **Symmetric Clustering** - In this, two or more hosts are running applications, and they are monitoring each other. This mode is obviously more efficient, as it uses all of the available hardware.
- **Parallel Clustering** - Parallel clusters allow multiple hosts to access the same data on the shared

storage. Because most operating systems lack support for this simultaneous data access by multiple hosts, parallel clusters are usually accomplished by special versions of software and special releases of applications.

Clustered technology is rapidly changing. Clustered system use and features should expand greatly as **Storage Area Networks (SANs)**. SANs allow easy attachment of multiple hosts to multiple storage units. Current clusters are usually limited to two or four hosts due to the complexity of connecting the hosts to shared storage.

OS OVERVIEW

Operating System is software that works as an interface between a user and the computer hardware. The primary **objective of an operating system** is to make computer system convenient to use and to utilize computer hardware in an efficient manner. The operating system performs the basic tasks such as receiving input from the keyboard, processing instructions and sending output to the screen.

The Software is the Non-Touchable Parts of the Computer, and Software's are those which are used for Performing an Operation So that Software's are just used for Making an Application but hardware's are those which are used for Performing an Operation. Operating system is software that is required in order to run application programs and utilities. It works as a bridge to perform better interaction between application programs and hardware of the computer. Various types of operating systems' are UNIX, MS-DOS, MS-Windows - 98/XP/Vista, WindowsNT/2000, OS/2 and Mac OS.

Operating system manages overall activities of a computer and the input/output devices attached to the computer. It is the first software you see when you turn on the computer, and the last software you see when the computer is turned off. It is the software that enables all the programs you use. At the simplest level, an operating system does two things:

The first, it manages the hardware and software resources of the computer system. These resources include the processor, memory, disk space, etc. The second, it provides a stable, consistent way for applications to deal with the hardware without having-to know all the details of the hardware.

The first task is very important i.e. managing the hardware and software resources, as various processes compete to each other for getting the CPU time and memory space to complete the task. In this regard; the operating system acts as a manager to allocate the available resources to 'satisfy the requirements of each process.

The second task i.e. providing a consistent application interface is especially important. A consistent application program interface (API) allows a user (or S/W developer) to write an application program on any computer and to run this program on another computer, even if the hardware configuration is different like as amount of memory, type of CPU or storage disk. It shields the user of

the machine from the low-level details of the machine's operation and provides frequently needed facilities. When you turn on the computer, the operating system program is loaded into the main memory. This program is called the kernel. Once initialized, the system program is prepared to run the user programs and permits them to use the hardware efficiently. Windows 98/XP is an excellent example that supports different types of hardware configurations from thousands of vendors and accommodates thousands of different I/O devices like printers, disk drives, scanners and cameras.

Operating systems may be classified based on if multiple tasks can be performed simultaneously, and if the system can be used by multiple users. It can be termed as single-user or multiuser OS, and single-tasking or multi-tasking OS. A multi-user system must be multi-tasking. MS-DOS and Windows 3x are examples of single user operating system. Whereas UNIX is an example of multi-user and multitasking operating system.

Classification of Operating systems:

- **Multi-user** : Allows two or more users to run programs at the same time. Some operating systems permit hundreds or even thousands of concurrent users.
- **Multiprocessing** : Supports running a program on more than one CPU.
- **Multitasking** : Allows more than one program to run concurrently.
- **Multithreading** : Allows different parts of a single program to run concurrently.
- **Real time** : Responds to input instantly. General-purpose operating systems, such as DOS and UNIX, are not real-time.

For Example if we want to Perform Some Paintings on the Screen, then we must use the Application Software as Paint and Hardware as a Mouse for Drawing an Object. But how the System knows what to do when Mouse Moves on the Screen and When the Mouse Draws a Line on the System so that Operating System is Necessary which Interact between or which Communicates with the Hardware and the Software.

Most Popular Desktop Operating Systems:

The three most popular types of operating systems for personal and business computing include Linux, Windows and Mac.

Windows - Microsoft Windows is a family of operating systems for personal and business computers. Windows dominates the personal computer world, offering a graphical user interface (GUI), virtual memory management, multitasking, and support for many peripheral devices.

Mac - Mac OS is the official name of the Apple Macintosh operating system. Mac OS features a graphical user interface (GUI) that utilizes windows, icons, and all applications that run on a Macintosh computer have a similar user interface.

Linux - Linux is a freely distributed open source operating system that runs on a number of hardware platforms. The Linux kernel was developed mainly by Linus Torvalds and it is based on Unix.

In the same way that a desktop OS controls your desktop or laptop computer, a mobile operating system is the software platform on top of which other programs can run on mobile devices, however, these systems are designed specifically to run on mobile devices such as mobile phones, smartphones, PDAs, tablet computers and other handhelds. The mobile OS is responsible for determining the functions and features available on your device, such as thumb wheel, keyboards, WAP, synchronization with applications, email, text messaging and more. The mobile OS will also determine which third-party applications (mobile apps) can be used on your device.

OBJECTIVES OF AN OPERATING SYSTEM

The objectives of an operating system are:

1. Convenience

The primary objective of an OS is to make the computer system **easy and convenient to use**. It hides the complexity of hardware and provides a user-friendly interface.

2. Efficiency

The OS ensures that **system resources** (CPU, memory, I/O devices) are used **efficiently**. It manages resources to achieve maximum performance and throughput.

3. Ability to Evolve

The OS should be designed in such a way that it can **adapt to new requirements**. It should allow **new features, hardware, and services** to be added without affecting existing functionality.

OPERATING SYSTEM FUNCTIONS

Operating system performs the following functions:

- 1. Processor management** which involves putting the tasks into order and pairing them into manageable size before they go to the CPU.
- 2. Memory management** which coordinates data to and from RAM (random-access memory) and determines the necessity for virtual memory.
- 3. Device management** which provides interface between connected devices.
- 4. Storage management** which directs permanent data storage.
- 5. Production and Security.**
- 6. Application** which allows standard communication between software and your computer.
- 7. User interface** which allows you to communicate with your computer.

Process Management:

A **program** does nothing unless its instructions are executed by a CPU. A program in execution, is a process. Example:

- A time-shared user program such as a compiler is a process.
- A word-processing program being run by an individual user on a PC is a process. A system task, such as sending output to a printer, can also be a process .
- A process needs certain resources-including CPU time, memory, files, and I/O devices-to accomplish its task

A **process** is the unit of work in a system. A system consists of a collection of processes, some of which are operating-system processes (those that execute system code) and the rest of which are user processes (those that execute user code). All these processes can potentially execute concurrently—by multiplexing on a single CPU, for example.

The operating system is responsible for the following activities in connection with process management:

- Scheduling processes and threads on the CPUs
- Creating and deleting both user and system processes
- Suspending and resuming processes
- Providing mechanisms for process synchronization
- Providing mechanisms for process communication

Memory Management:

The main memory is generally the only large storage device that the CPU is able to address and access directly. For example, for the CPU to process data from disk, those data must first be transferred to main memory by CPU-generated I/O calls. In the same way, instructions must be in memory for the

CPU to execute them. For a program to be executed, it must be mapped to absolute addresses and loaded into memory. As the program executes, it accesses program instructions and data from memory by generating these absolute addresses. Eventually, the program terminates, its memory space is declared available, and the next program can be loaded and executed.

The operating system is responsible for the following activities in connection with memory management:

- Keeping track of which parts of memory are currently being used and who is using them
- Deciding which processes (or parts of processes) and data to move into and out of memory
- Allocating and deallocating memory space as needed

Storage Management:

To make the computer system convenient for users, the operating system provides a uniform, logical view of information storage. The operating system abstracts from the physical properties of its storage devices to define a logical storage unit, the **file**. The operating system maps files onto physical media and accesses these files via the storage devices.

File-System Management:

File management is one of the most visible components of an operating system. Computers can store information on several different types of physical media. Magnetic disk, optical disk, and magnetic tape are the most common. Each of these media has its own characteristics and physical organization. Each medium is controlled by a device, such as a disk drive or tape drive, that also has its own unique characteristics. These properties include access speed, capacity, data-transfer rate, and access method (sequential or random).

A file is a collection of related information defined by its creator. Commonly, files represent programs (both source and object forms) and data. Data files may be numeric, alphabetic, alphanumeric, or binary. Files may be free-form (for example, text files), or they may be formatted rigidly (for example, fixed fields).

Clearly, the concept of a file is an extremely general one. The operating system implements the abstract concept of a file by managing mass-storage media, such as tapes and disks, and the devices that control them. In addition, files are normally organized into directories to make them easier to use. Finally, when multiple users have access to files, it may be desirable to control which user may access a file and how that user may access it (for example, read, write, append).

The operating system is responsible for the following activities in connection with file management:

- Creating and deleting files
- Creating and deleting directories to organize files
- Supporting primitives for manipulating files and directories
- Mapping files onto secondary storage
- Backing up files on stable (nonvolatile) storage media

Mass-Storage Management:

The main memory is too small to accommodate all data and programs, and because the data that it holds are lost when power is lost, the computer system must provide secondary storage to back up main memory. Most modern computer systems use disks as the principal on-line storage medium for both programs and data. Most programs—including compilers, assemblers, word processors, editors, and formatters—are stored on a disk until loaded into memory. They then use the disk as both the source and destination of their processing. Hence, the proper management of disk storage is of central importance to a computer system. The operating system is responsible for the following activities in connection with disk management:

- Free-space management
- Storage allocation

- Disk scheduling

Because secondary storage is used frequently, it must be used efficiently. The entire speed of operation of a computer may hinge on the speeds of the disk subsystem and the algorithms that manipulate that subsystem.

Caching:

Caching is an important principle of computer systems. Here's how it works. Information is normally kept in some storage system (such as main memory). As it is used, it is copied into a faster storage system—the cache—on a temporary basis. When we need a particular piece of information, we first check whether it is in the cache. If it is, we use the information directly from the cache. If it is not, we use the information from the source, putting a copy in the cache under the assumption that we will need it again soon. In a hierarchical storage structure, the same data may appear in different levels of the storage system. For example, suppose that an integer *A* that is to be incremented by 1 is located in file *B*, and file *B* resides on magnetic disk.

The increment operation proceeds by first issuing an I/O operation to copy the disk block on which *A* resides to main memory. This operation is followed by copying *A* to the cache and to an internal register. Thus, the copy of *A* appears in several places: on the magnetic disk, in main memory, in the cache, and in an internal register. Once the increment takes place in the internal register, the value of *A* differs in the various storage systems. The value of *A* becomes the same only after the new value of *A* is written from the internal register back to the magnetic disk.

I/O Systems:

One of the purposes of an operating system is to hide the peculiarities of specific hardware devices from the user. For example, in UNIX, the peculiarities of I/O devices are hidden from the bulk of the operating system itself by the **I/O subsystem**.

The I/O subsystem consists of several components:

- A memory-management component that includes buffering, caching, and spooling
- A general device-driver interface
- Drivers for specific hardware devices

Only the device driver knows the peculiarities of the specific device to which it is assigned.

Protection and Security:

Protection is any mechanism for controlling access of processes or users to resources defined by the OS. **Security** – defense of the system against internal and external attacks

- Huge range, including denial-of-service, worms, viruses, identity theft, theft of service
- Systems generally first distinguish among users, to determine who can do what

- User identities (**user IDs**, security IDs) include name and associated number, one per user
- User ID then associated with all files, processes of that user to determine access control
- Group identifier (**group ID**) allows set of users to be defined and controls managed, then also associated with each process, file
- **Privilege escalation** allows user to change to effective ID with more rights

Providing interface:

It is used in order that user interface acts with a computer mutually. User interface controls how you input data and instruction and how information is displayed on screen. The operating system offers two types of the interface to the user:

- a. Graphical-line interface: It interacts with of visual environment to communicate with the computer. It uses windows, icons, menus and other graphical objects to issues commands.
- b. Command-line interface: it provides an interface to communicate with the computer by typing commands.

OPERATING-SYSTEM OPERATIONS

In modern operating systems are **interrupt driven**. If there are no processes to execute, no I/O devices to service, and no users to whom to respond, an operating system will sit quietly, waiting for something to happen. Events are almost always signaled by the occurrence of an interrupt or a trap.

A **trap** (or an **exception**) is a software-generated interrupt caused either by an error (for example, division by zero or invalid memory access) or by a specific request from a user program that an operating-system service be performed. The interrupt-driven nature of an operating system defines that system's general structure.

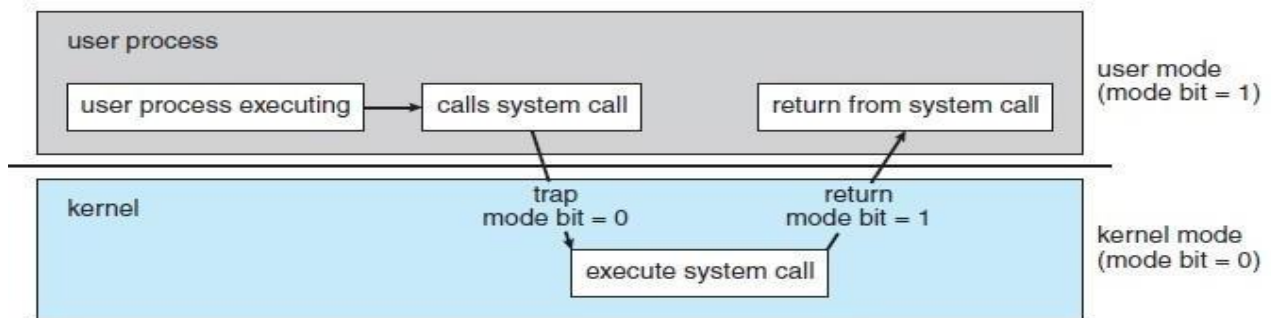
For each type of interrupt, separate segments of code in the operating system determine what action should be taken. An interrupt service routine is provided to deal with the interrupt. Since the operating system and the users share the hardware and software resources of the computer system, we need to make sure that an error in a user program could cause problems only for the one program running. With sharing, many processes could be adversely affected by a bug in one program.

For example, if a process gets stuck in an infinite loop, this loop could prevent the correct operation of many other processes. More subtle errors can occur in a multiprogramming system, where one erroneous program might modify another program, the data of another program, or even the operating system itself. Without protection against these sorts of errors, either the computer must execute only one process at a time or all output must be suspect. A properly designed operating system must ensure that an incorrect (or malicious) program cannot cause other programs to execute incorrectly.

Dual-Mode and Multimode Operation:

In order to ensure the proper execution of the operating system, we must be able to distinguish

between the execution of operating-system code and user defined code. The approach taken by most computer systems is to provide hardware support that allows us to differentiate among various modes of execution.



Transition from user to kernel mode.

At the very least, we need two separate **modes** of operation:

1. **User mode** and
2. **Kernel mode** (also called **supervisor mode**, **system mode**, or **privileged mode**).

A bit, called the **mode bit**, is added to the hardware of the computer to indicate the current mode: kernel (0) or user (1). With the mode bit, we can distinguish between a task that is executed on behalf of the operating system and one that is executed on behalf of the user. When the computer system is executing on behalf of a user application, the system is in user mode.

However, when a user application requests a service from the operating system (via a system call), the system must transition from user to kernel mode to fulfill the request. This is shown in above Figure. As we shall see, this architectural enhancement is useful for many other aspects of system operation as well.

At system boot time, the hardware starts in kernel mode. The operating system is then loaded and starts user applications in user mode. Whenever a trap or interrupt occurs, the hardware switches from user mode to kernel mode (that is, changes the state of the mode bit to 0). Thus, whenever the operating system gains control of the computer, it is in kernel mode. The system always switches to user mode (by setting the mode bit to 1) before passing control to a user program.

Timer:

We must ensure that the operating system maintains control over the CPU. We cannot allow a user program to get stuck in an infinite loop or to fail to call system services and never return control to the operating system. To accomplish this goal, we can use a **timer**.

A timer can be set to interrupt the computer after a specified period. The period may be fixed (for example, 1/60 second) or variable (for example, from 1 millisecond to 1 second). A **variable timer** is

generally implemented by a fixed-rate clock and a counter.

The operating system sets the counter. Every time the clock ticks, the counter is decremented. When the counter reaches 0, an interrupt occurs. For instance, a 10-bit counter with a 1-millisecond clock allows interrupts at intervals from 1 millisecond to 1,024 milliseconds, in steps of 1 millisecond. Before turning over control to the user, the operating system ensures that the timer is set to interrupt. If the timer interrupts, control transfers automatically to the operating system, which may treat the interrupt as a fatal error or may give the program more time. Clearly, instructions that modify the content of the timer are privileged.

HISTORY OF OPERATING SYSTEMS

The History of operating systems is directly dependent to the development of computer systems and how users use them. Here is a quick tour of computing systems through the past fifty years in the timeline.

Early Evolution

- 1945: ENIAC, Moore School of Engineering, University of Pennsylvania.
- 1949: EDSAC and EDVAC
- 1949 BINAC - a successor to the ENIAC
- 1951: UNIVAC by Remington
- 1952: IBM 701
- 1956: The interrupt
- 1954-1957: FORTRAN was developed

Operating Systems by the late 1950s

By the late 1950s Operating systems were well improved and started supporting following usages :

- It was able to Single stream batch processing
- It could use Common, standardized, input/output routines for device access
- Program transition capabilities to reduce the overhead of starting a new job was added
- Error recovery to clean up after a job terminated abnormally was added.
- Job control languages that allowed users to specify the job definition and resource requirements were made possible.

Operating Systems In 1960s

- 1961: The dawn of minicomputers
- 1962 Compatible Time-Sharing System (CTSS) from MIT
- 1963 Burroughs Master Control Program (MCP) for the B5000 system

- 1964: IBM System/360
- 1960s: Disks become mainstream
- 1966: Minicomputers get cheaper, more powerful, and really useful
- 1967-1968: The mouse
- 1964 and onward: Multics
- 1969: The UNIX Time-Sharing System from Bell Telephone Laboratories

Supported OS Features by 1970s

- Multi User and Multi tasking was introduced.
- Dynamic address translation hardware and Virtual machines came into picture.
- Modular architectures came into existence.
- Personal, interactive systems came into existence.

Accomplishments after 1970

- 1971: Intel announces the microprocessor
- 1972: IBM comes out with VM: the Virtual Machine Operating System
- 1973: UNIX 4th Edition is published
- 1973: Ethernet
- 1974 The Personal Computer Age begins
- 1974: Gates and Allen wrote BASIC for the Altair
- 1976: Apple II
- August 12, 1981: IBM introduces the IBM PC
- 1983 Microsoft begins work on MS-Windows
- 1984 Apple Macintosh comes out
- 1990 Microsoft Windows 3.0 comes out
- 1991 GNU/Linux
- 1992 The first Windows virus comes out
- 1993 Windows NT
- 2007: iOS
- 2008: Android OS

And as the research and development work still goes on, with new operating systems being developed and existing ones getting improved and modified to enhance the overall user experience, making operating systems fast and efficient like never before.

EVOLUTION OF OS

OS were developed mainly to facilitate the use of the hardware and to bring it to the best advantage. Here is the brief sketch of the evolutionary path of OS development.

1. Mainframe Systems (1940s – 1960s)

Mainframe systems are **large and powerful computer systems** designed to handle **high-volume data processing** and support a **large number of users simultaneously**. They are widely used in **banks, airlines, insurance companies, and government organizations** where **reliability and performance are critical**. Mainframe systems allow **hundreds or thousands of users** to access the system at the same time through terminals and are optimized for **maximum throughput and efficient resource utilization**. Early mainframe systems used **batch processing**, and later adopted **multiprogramming and time-sharing** to improve **CPU utilization** and enable **interactive computing**. Mainframe operating systems provide **strong security, fault tolerance, and continuous operation** without interruption.

However, mainframe systems have disadvantages such as **very high installation and maintenance costs, large physical size, and high power consumption**. They also require **specialized skills** to operate and maintain and offer **less flexibility** compared to modern personal computers and distributed systems.

2. Batch Processing Operating System: (1950s – 1960s)

To overcome the inefficiencies of early computers, **batch operating systems** were introduced. In batch systems, jobs from different users were **collected into batches** and stored on input devices such as **punch cards or magnetic tapes**. Jobs with similar requirements were grouped together and executed **one after another without user interaction**.

The operating system included a small program called a **resident monitor**, which controlled job sequencing and execution. Once a job finished, the next job in the batch was automatically loaded and executed. This reduced CPU idle time and improved system utilization.

However, batch operating systems had several drawbacks. There was **no interaction between the user and the program during execution**, and users had to wait a long time to receive results. If a job contained an error, it could only be detected after execution, leading to increased turnaround time.

3. Multiprogramming Operating System: (1960s)

With advancements in hardware such as larger memory and faster processors, **multiprogramming operating systems** were developed. In these systems, **multiple programs were kept in the main memory at the same time**. The CPU was switched among these programs to ensure that it was always busy.

When one program was waiting for input/output operations, the CPU was allocated to another program. This approach significantly improved **CPU utilization and system throughput**. Multiprogramming introduced important concepts such as **process management, memory**

management, and CPU scheduling. This stage marked a major improvement in operating system design, as it allowed better use of system resources and improved overall performance compared to batch systems.

4. Time-Sharing/multitasking Operating Systems (1970s)

Time-sharing or multitasking operating systems were designed to support **interactive computing** by allowing multiple users to use the computer system simultaneously. In these systems, each user interacts with the system through terminals, and the **CPU time is divided into small units called time slices**. Each user process is allocated a time slice in a **round-robin fashion**, creating the **illusion that every user has exclusive access to the system**.

Time-sharing operating systems emphasize **quick response time** rather than maximum throughput. Users can enter commands, edit files, and receive **immediate feedback**, which greatly improves user experience. These systems were widely used in **universities and research institutions**. Time-sharing introduced important concepts such as **process synchronization, file systems, and user-level interaction**. However, time-sharing operating systems have disadvantages such as **high overhead due to frequent context switching**. System performance may degrade when the **number of users increases**, leading to longer response times. They also require **complex scheduling and memory management**, making system design more complicated.

5. Multiprocessor Operating Systems (1980s)

Multiprocessor operating systems are also known as parallel OS or tightly coupled OS. Such operating systems have more than one processor in close communication that sharing the computer bus, the clock and sometimes memory and peripheral devices. It executes multiple jobs at same time and makes the processing faster. Multiprocessor systems have three main advantages:

□ **Increased throughput:** By increasing the number of processors, the system performs more work in less time.

The speed-up ratio with **N** processors is less than **N**.

□ **Economy of scale:** Multiprocessor systems can save more money than multiple single-processor systems, because they can share peripherals, mass storage, and power supplies.

□ **Increased reliability:** If one processor fails to done its task, then each of the remaining processors must pick up a share of the work of the failed processor. The failure of one processor will not halt the system, only slow it down.

The ability to continue providing service proportional to the level of surviving hardware is called **graceful degradation**. Systems designed for graceful degradation are called **fault tolerant**. The multiprocessor operating systems are classified into two categories:

1. Symmetric multiprocessing system
2. Asymmetric multiprocessing system

In symmetric multiprocessing system, each processor runs an identical copy of the operating system, and these copies communicate with one another as needed. In asymmetric multiprocessing system, a processor is called master processor that controls other processors called slave processor. Thus, it establishes master-slave relationship. The master processor schedules the jobs and manages the memory for entire system.

6. Distributed Operating Systems (1980s – 1990s)

In distributed system, the different machines are connected in a network and each machine has its own processor and own local memory. In this system, the operating systems on all the machines work together to manage the collective network resource. It can be classified into two categories:

1. Client-Server systems

2. Peer-to-Peer systems

Advantages of distributed systems.

- ☐ Resources Sharing
- ☐ Reliability
- ☐ Requires networking infrastructure.
- ☐ Computation speed up – load sharing
- ☐ Communications
- ☐ Local area networks (LAN) or Wide area networks

7. Desktop Systems/Personal Computer Systems (1980s – 1990s)

The PC operating system is designed for maximizing user convenience and responsiveness. This system is neither multi-user nor multitasking. These systems include PCs running Microsoft Windows and the Apple Macintosh. The MS-DOS operating system from Microsoft has been superseded by multiple flavors of Microsoft Windows and IBM has upgraded MS-DOS to the OS/2 multitasking system. The Apple Macintosh operating system has been ported to more advanced hardware, and now includes new features such as virtual memory and multitasking.

8. Real-Time Operating Systems (RTOS) (1990s – Present)

A real-time operating system (RTOS) is a multitasking operating system intended for applications with fixed deadlines (real-time computing). Such applications include some small embedded systems, automobile engine controllers, industrial robots, spacecraft, industrial control, and some large-scale computing systems. The real time operating system can be classified into two categories:

1. hard real time system and 2. soft real time system.

☐ A **hard real-time** system guarantees that critical tasks be completed on time. This goal requires that all delays in the system be bounded, from the retrieval of stored data to the time that it takes the operating system to finish any request made of it. Such time constraints dictate the facilities that are available in hard real-time systems.

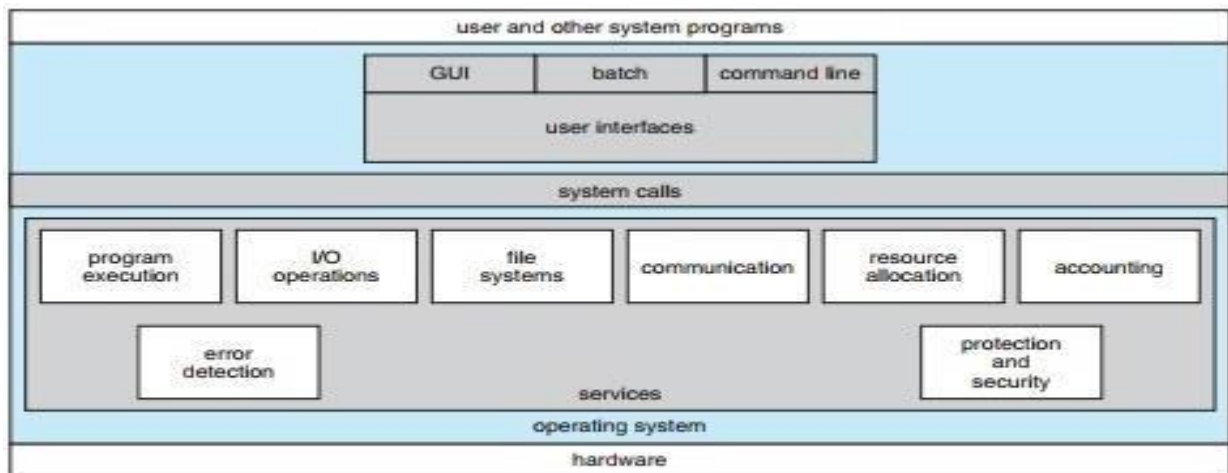
☐ A **soft real-time** system is a less restrictive type of real-time system. Here, a critical real-time task gets priority over other tasks and retains that priority until it completes. Soft real time system can be mixed with other types of systems. Due to less restriction, they are risky to use for industrial control and robotics.

OPERATING SYSTEM SERVICES

An Operating System provides services to both the users and to the programs.

- It provides programs an environment to execute.
- It provides users the services to execute the programs in a convenient manner.

The specific services provided, of course, differ from one operating system to another, but we can identify common classes. These operating system services are provided for the convenience of the programmer, to make the programming task.



A view of operating system services.

The above Figure shows one view of the various operating-system services and how they interrelate.

- User Interface
- Program Execution
- I/O operations
- File System manipulation
- Communication
- Error Detection
- Resource Allocation
- Accounting
- Protection and Security

One set of operating system services provides functions that are helpful to the user.

User interface:

Almost all operating systems have a **user interface (UI)**. This interface can take several forms.

- One is a **command-line interface (CLI)**, which uses text commands and a method for entering them (say, a keyboard for typing in commands in a specific format with specific options).

- Another is a **batch interface**, in which commands and directives to control those commands are entered into files, and those files are executed.

Most commonly, a **graphical user interface (GUI)** is used. Here, the interface is a window system with a pointing device to direct I/O, choose from menus, and make selections and a keyboard to enter text. Some systems provide two or all three of these variations.

Program execution:

The system must be able to load a program into memory and to run that program. The program must be able to end its execution, either normally or abnormally (indicating error).

I/O Operation:

An I/O subsystem comprises of I/O devices and their corresponding driver software. Drivers hide the peculiarities of specific hardware devices from the users. An Operating System manages the communication between user and device drivers.

- I/O operation means read or write operation with any file or any specific I/O device.
- Operating system provides the access to the required I/O device when required.

File system manipulation:

A file represents a collection of related information. Computers can store files on the disk (secondary storage), for long-term storage purpose. Examples of storage media include magnetic tape, magnetic disk and optical disk drives like CD, DVD. Each of these media has its own properties like speed, capacity, data transfer rate and data access methods.

A file system is normally organized into directories for easy navigation and usage. These directories may contain files and other directions.

Following are the major activities of an operating system with respect to file management –

- Program needs to read a file or write a file.
- The operating system gives the permission to the program for operation on file.
- Permission varies from read-only, read-write, denied and so on.
- Operating System provides an interface to the user to create/delete files.
- Operating System provides an interface to the user to create/delete directories.
- Operating System provides an interface to create the backup of file system.

Communication:

In case of distributed systems which are a collection of processors that do not share memory, peripheral devices, or a clock, the operating system manages communications between all the processes. Multiple processes communicate with one another through communication lines in the network. The OS handles routing and connection strategies, and the problems of contention and security. Following are the major activities of an operating system with respect to communication

- Two processes often require data to be transferred between them
- Both the processes can be on one computer or on different computers, but are connected through a computer network.
- Communication may be implemented by two methods, either by Shared Memory or by Message Passing.

Error detection:

Errors can occur anytime and anywhere. An error may occur in CPU, in I/O devices or in the memory hardware. Following are the major activities of an operating system with respect to error handling

- The OS constantly checks for possible errors.
- The OS takes an appropriate action to ensure correct and consistent computing.

Another set of operating system functions exists not for helping the user but rather for ensuring the efficient operation of the system itself. Systems with multiple users can gain efficiency by sharing the computer resources among the users.

Resource Allocation

In case of multi-user or multi-tasking environment, resources such as main memory, CPU cycles and files storage are to be allocated to each user or job. Following are the major activities of an operating system with respect to resource management

- The OS manages all kinds of resources using schedulers.
- CPU scheduling algorithms are used for better utilization of CPU.

Accounting:

We want to keep track of which users use how much and what kinds of computer resources. This record keeping may be used for accounting (so that users can be billed) or simply for accumulating usage statistics. Usage statistics may be a valuable tool for researchers who wish to reconfigure the system to improve computing services.

Protection and Security:

Considering a computer system having multiple users and concurrent execution of multiple processes, the various processes must be protected from each other's activities. Protection refers to a mechanism or a way to control the access of programs, processes, or users to the resources defined by a computer system. Following are the major activities of an operating system with respect to protection

- The OS ensures that all access to system resources is controlled.
- The OS ensures that external I/O devices are protected from invalid access attempts.
- The OS provides authentication features for each user by means of passwords.

THE OPERATING SYSTEM AS RESOURCE MANAGER

- A computer is a set of resources for the movement, storage, and processing of data and for the control of these functions.
- The OS is responsible for managing these resources.
- The OS functions in the same way as ordinary computer software; that is, it is a program or suite of programs executed by the processor.
- The OS frequently relinquishes control and must depend on the processor to allow it to regain control.
- The OS directs the processor in the use of the other system resources and in the timing of its execution of other programs.
- A portion of the OS is in main memory. This includes the **kernel**, or **nucleus**, the most frequently used functions in the OS and, at a given time, other portions of the OS currently in use.
- The remainder of main memory contains user programs and data.
- The memory management hardware in the processor and the OS jointly control the allocation of main memory.

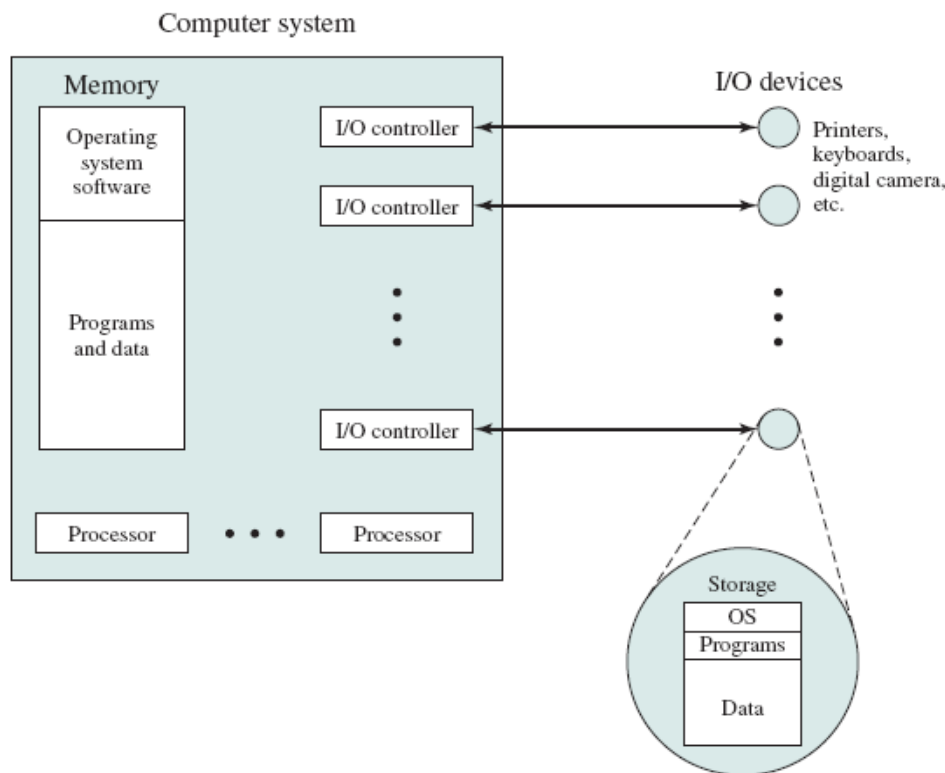


Figure 2.2 The Operating System as Resource Manager

A portion of the OS is in main memory. This includes the **kernel**, or **nucleus**, which contains

- The most frequently used functions in the OS and, at a given time, other portions of the OS currently in use.
- The remainder of main memory contains user programs and data.
- The memory management hardware in the processor and the OS jointly control the allocation of main memory.

USER AND OPERATING SYSTEM INTERFACE

We mentioned earlier that there are several ways for users to interface with the operating system. Here, we discuss two fundamental approaches.

One provides a **command-line interface, or command interpreter**, that allows users to directly enter commands to be performed by the operating system.

The other allows users to interface with the operating system- a GUI -**Graphical User Interfaces**.

Command Line Interpreter:

Some operating systems include the command interpreter in the kernel. Some, such as the popular Windows and Linux operating systems, use the command interpreter as a special program that runs when a user logs on or a job is initiated.

- In Windows, this is the MS-DOS prompt.
- Linux has more options. The command interpreter in Linux is known as a shell. The most commonly used shell is the Bash shell, but others such as the Korn shell, C shell, and Bourne shell exist. Most shells provide similar functionality, personal preference usually dictates which shell is best.
- The main function of the command utility is to receive and execute the next user generated command.
- Many commands are intended to manipulate files.
- Operating systems such as UNIX implements commands through system programs. Often these programs are stored as text files, which allow programmers to add additional functionality to the utility.

Thus, the UNIX command to delete a file

Root rm file.txt

would search for a file called rm, load the file into memory, and execute it with the parameter file.txt.

Graphical User Interfaces – GUI:

Here, rather than entering commands directly via a command-line interface, users employ a mouse-based window and- menu system characterized by a **desktop** metaphor.

The user moves the mouse to position its pointer on images, or **icons**, on the screen (the desktop) that represent programs, files, directories, and system functions. Depending on the mouse pointer's location, clicking a button on the mouse can invoke a program, select a file or directory—known as a **folder**—or pull down a menu that contains commands.

Because a mouse is impractical for most mobile systems, smartphones and handheld tablet computers typically use a touchscreen interface. Here, users interact by making **gestures** on the touchscreen—for example, pressing and swiping fingers across the screen

Choice of Interface:

The choice of whether to use a command-line or GUI interface is mostly one of personal preference.

System administrators who manage computers and power users who have deep knowledge of a system frequently use the command-line interface. For them, it is more efficient, giving them faster access to the activities they need to perform.

Indeed, on some systems, only a subset of system functions is available via the GUI, leaving the less common tasks to those who are command-line knowledgeable. Further, command line interfaces usually make repetitive tasks easier, in part because they have their own programmability.

For example, if a frequent task requires a set of command-line steps, those steps can be recorded into a file, and that file can be run just like a program.

A GUI provides a mouse-based windows and menu system as an interface. Users of Windows are more likely to use the GUI rather than the command line interface of MS-DOS, while UNIX users generally prefer using the command line interface of the shell rather than the GUI.

SYSTEM CALLS

System calls provide an interface to the services made available by an operating system.

Application developers often do not have direct access to the system calls, but can access them through an application programming interface (API). The functions that are included in the API invoke the actual system calls. By using the API, certain benefits can be gained:

- Portability: as long a system supports an API, any program using that API can compile and run.
- Ease of Use: using the API can be significantly easier than using the actual system call.

These calls are generally available as routines written in C and C++, although certain low-level tasks (for example, tasks where hardware must be accessed directly) may have to be written using assembly-language instructions.

Before we discuss how an operating system makes system calls available, let's first use an example to illustrate how system calls are used: writing a simple program to read data from one file and copy them to another file.

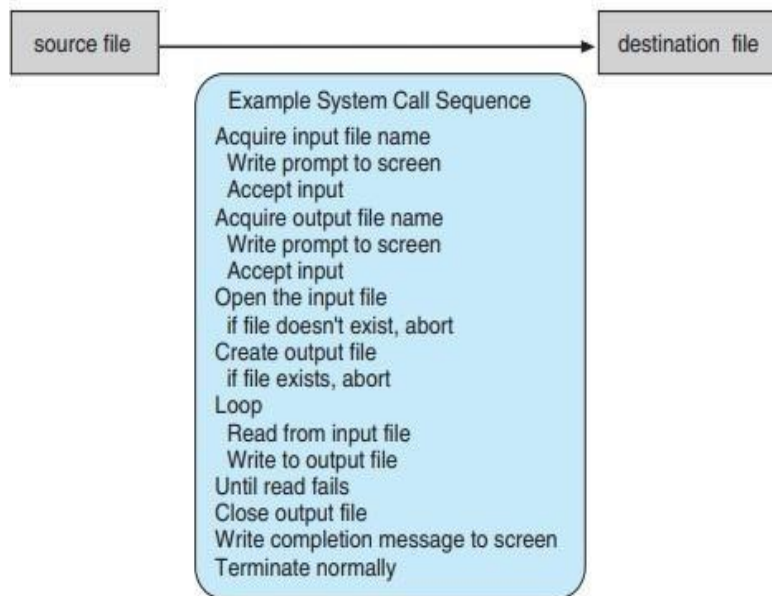
The first input that the program will need is the names of the two files: the input file and the output file. These names can be specified in many ways, depending on the operating-system design. One approach is for the program to ask the user for the names.

In an interactive system, this approach will require a sequence of system calls, first to write a prompting message on the screen and then to read from the keyboard the characters that define the two files.

On mouse-based and icon-based systems, a menu of file names is usually displayed in a window. The user can then use the mouse to select the source name, and a window can be opened for the

destination name to be specified. This sequence requires many I/O system calls.

Once the two file names have been obtained, the program must open the input file and create the output file. Each of these operations requires another system call. Possible error conditions for each operation can require additional system calls. When the program tries to open the input file, for example, it may find that there is no file of that name or that the file is protected against access.



Example of how system calls are used.

In these cases, the program should print a message on the console (another sequence of system calls) and then terminate abnormally (another system call). If the input file exists, then we must create a new output file. We may find that there is already an output file with the same name. This situation may cause the program to abort (a system call), or we may delete the existing file (another system call) and create a new one (yet another system call).

Another option, in an interactive system, is to ask the user (via a sequence of system calls to output the prompting message and to read the response from the terminal) whether to replace the existing file or to abort the program.

When both files are set up, we enter a loop that reads from the input file (a system call) and writes to the output file (another system call). Each read and write must return status information regarding various possible error conditions.

On input, the program may find that the end of the file has been reached or that there was a hardware failure in the read (such as a parity error). The write operation may encounter various errors, depending on the output device (for example, no more disk space).

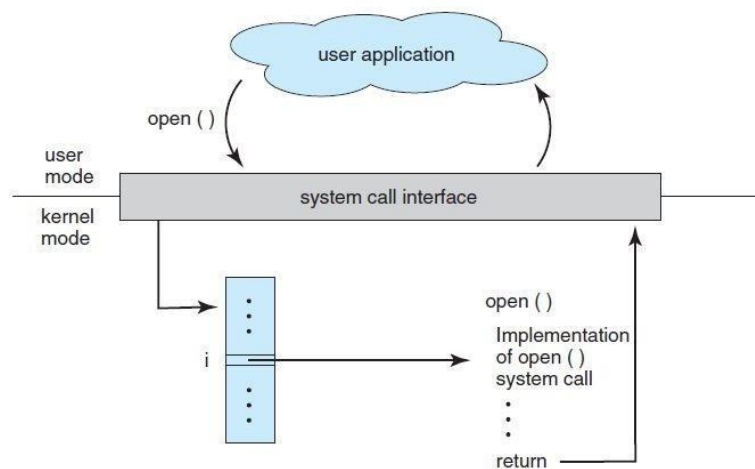
Finally, after the entire file is copied, the program may close both files (another system call), write a message to the console or window (more system calls), and finally terminate normally (the final system call). This system-call sequence is shown in above Figure.

Frequently, systems execute thousands of system calls per second. Most programmers never see this level of detail, however. Typically, application developers design programs according to an **application programming interface (API)**.

- Programming interface to the services provided by the OS
- Typically written in a high-level language (C or C++)
- Mostly accessed by programs via a high-level Application Program Interface (API)

Three most common APIs are

1. Win32 API for Windows,
2. POSIX API (all versions of UNIX, Linux, and Mac OS X), and
3. Java API for the Java virtual machine (JVM)

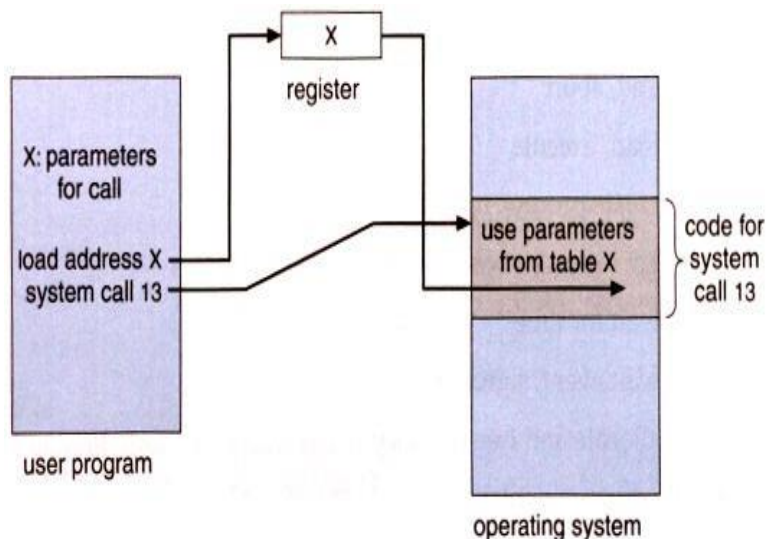


The handling of a user application invoking the `open()` system call.

Thus, most of the details of the operating-system interface are hidden from the programmer by the API and are managed by the run-time support library. The relationship between an API, the system-call interface, and the operating system is shown in Figure, which illustrates how the operating system handles a user application invoking the `open()` system call.

System calls Parameters:

Three general methods exist for passing parameters to the OS:



1. Parameters can be passed in registers.

2. When there are more parameters than registers, parameters can be stored in a block and the block address can be passed as a parameter to a register.

3. Parameters can also be pushed on or popped off the stack by the operating system.

TYPES OF SYSTEM CALLS

System calls can be grouped roughly into six major categories: process control, file manipulation, device manipulation, information maintenance, communications, and protection.

	Windows	Unix
Process Control	CreateProcess() ExitProcess() WaitForSingleObject()	fork() exit() wait()
File Manipulation	CreateFile() ReadFile() WriteFile() CloseHandle()	open() read() write() close()
Device Manipulation	SetConsoleMode() ReadConsole() WriteConsole()	ioctl() read() write()
Information Maintenance	GetCurrentProcessID() SetTimer() Sleep()	getpid() alarm() sleep()
Communication	CreatePipe() CreateFileMapping() MapViewOfFile()	pipe() shmget() mmap()
Protection	SetFileSecurity() InitializeSecurityDescriptor() SetSecurityDescriptorGroup()	chmod() umask() chown()

Process Control

- end, abort
- load, execute
- create process, terminate process
- get process attributes,
- set process attributes
- wait for time
- wait event, signal event
- allocate and free memory

File Management

- create file, delete file
- open, close
- read, write, reposition
- get file attributes, set file attributes

Device Management

- request device, release device
- read, write, reposition
- get device attributes,
- set device attributes
- logically attach or detach devices

Information maintenance

- request device, release device
- read, write, reposition
- get device attributes,
- set device attributes
- logically attach or detach devices

Communication

- create, delete comm. connection
- send, receive messages
- transfer status information
- attach or detach remote devices

Process Control:

A running program needs to be able to stop execution either normally or abnormally. When execution is stopped abnormally, often a dump of memory is taken and can be examined with a debugger.

File Management:

Some common system calls are *create*, *delete*, *read*, *write*, *reposition*, or *close*. Also, there is a need to determine the file attributes – *get* and *set* file attribute. Many times the OS provides an API to make these system calls.

Device Management:

Process usually requires several resources to execute, if these resources are available, they will be granted and control returned to the user process. These resources are also thought of as devices. Some are physical, such as a video card, and others are abstract, such as a file.

User programs *request* the device, and when finished they *release* the device. Similar to files, we can *read*, *write*, and *reposition* the device.

Information Management:

Some system calls exist purely for transferring information between the user program and the operating system. An example of this is *time*, or *date*.

The OS also keeps information about all its processes and provides system calls to report this information.

Communication:

There are two models of interprocess communication, the message-passing model and the shared memory model.

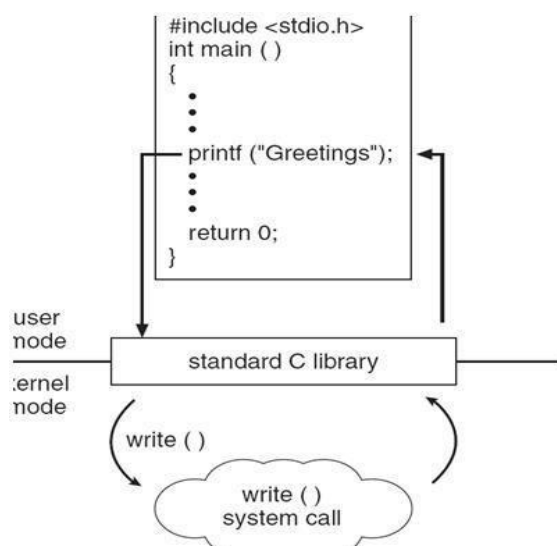
- Message-passing uses a common mailbox to pass messages between processes.
- Shared memory use certain system calls to create and gain access to create and gain access to regions of memory owned by other processes. The two processes exchange information by reading and writing in the shared data.

Protection

Protection provides a mechanism for controlling access to the resources provided by a computer system. Historically, protection was a concern only on multiprogrammed computer systems with several users. However, with the advent of networking and the Internet, all computer systems, from servers to mobile handheld devices, must be concerned with protection.

Typically, system calls providing protection include `set permission()` and `get permission()`, which manipulate the permission settings of resources such as files and disks. The `allow user()` and `deny user()` system calls specify whether particular users can—or cannot—be allowed access to certain resources.

Example of Standard C Library:

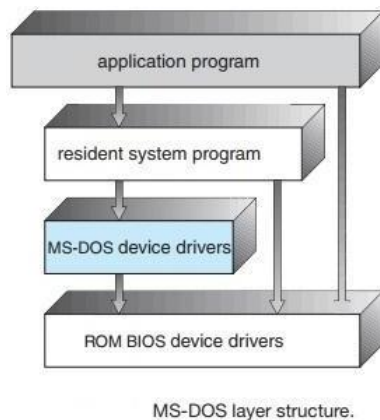


- The standard C library provides a portion of the system call interface for many version of UNIX and Linux.
- As an example, let's assume a C program invokes `printf()` statement.
- The C library intercepts this call and invokes the necessary system call(s) in the OS.
- The C library takes the value returned by `write()` and passes it back to the user program

OPERATING-SYSTEM STRUCTURE

A system as large and complex as a modern operating system must be engineered carefully if it is to function properly and be modified easily. A common approach is to partition the task into small components, or modules, rather than have one **monolithic** system. Each of these modules should be a well-defined portion of the system, with carefully defined inputs, outputs, and functions.

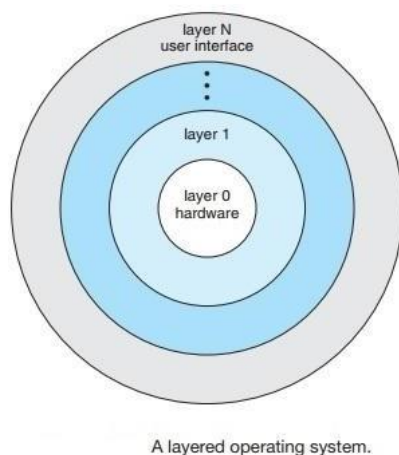
Simple Structure



Many operating systems do not have well-defined structures. Frequently, such systems started as small, simple, and limited systems and then grew beyond their original scope. MS-DOS is an example of such a system. It was originally designed and implemented by a few people who had no idea that it would become so popular. It was written to provide the most functionality in the least space, so it was not carefully divided into modules. Figure shows its structure.

In MS-DOS, the interfaces and levels of functionality are not well separated. For instance, application programs are able to access the basic I/O routines to write directly to the display and disk drives. Such freedom leaves MS-DOS vulnerable to errant (or malicious) programs, causing entire system crashes when user programs fail. Of course, MS-DOS was also limited by the hardware of its era. Because the Intel 8088 for which it was written provides no dual mode and no hardware protection, the designers of MS-DOS had no choice but to leave the base hardware accessible.

Layered Approach



A system can be made modular in many ways. One method is the **layered approach**, in which the operating system is broken into a number of layers (levels). The bottom layer (layer 0) is the hardware; the highest (layer N) is the user interface. This layering structure is depicted in Figure.

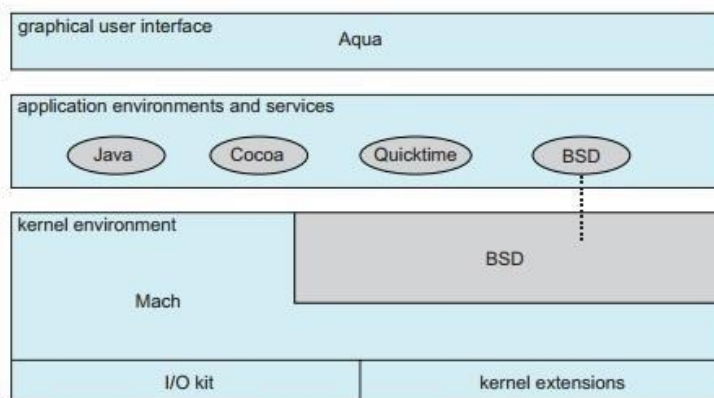
An operating-system layer is an implementation of an abstract object made up of data and the operations that can manipulate those data. A typical operating-system layer—say, layer *M*—consists of data structures

and a set of routines that can be invoked by higher-level layers. Layer *M*, in turn, can invoke operations on lower-level layers.

- The main advantage of the layered approach is simplicity of construction and debugging.
- The major difficulty with the layered approach involves appropriately defining the various layers.

Microkernels

In a **microkernel** (also known as μ -kernel) is the near-minimum amount of software that can provide the mechanisms needed to implement an **operating system (OS)**. These mechanisms include low-level address space management, thread management, and inter-process communication (IPC).



The Mac OS X structure.

We have already seen that as UNIX expanded, the kernel became large and difficult to manage. In the mid-1980s, researchers at Carnegie Mellon University developed an operating system called **Mach** that modularized the kernel using the **microkernel** approach.

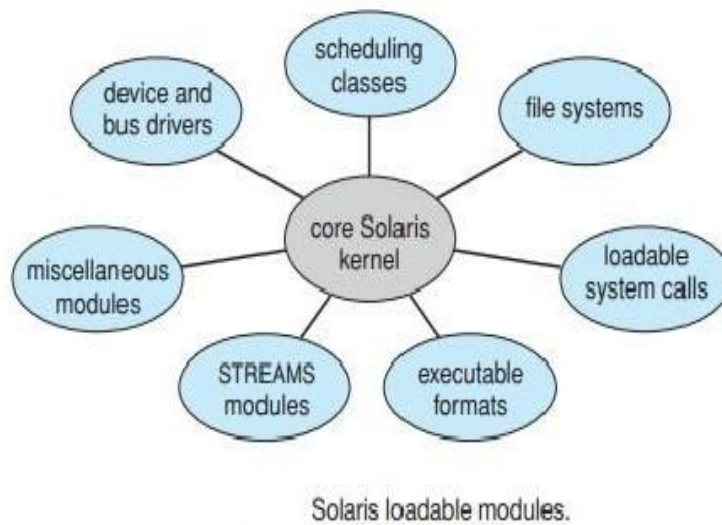
This method structures the operating system by removing all nonessential components from the kernel and implementing them as system and user-level programs. The result is a smaller kernel. There is little consensus regarding which services should remain in the kernel and which should be implemented in user space.

Typically, however, microkernels provide minimal process and memory management, in addition to a communication facility. Figure illustrates the architecture of a typical microkernel.

The main function of the microkernel is to provide communication between the client program and the various services that are also running in user space. Communication is provided through **message passing**. One benefit of the microkernel approach is that it makes extending the operating system easier.

Example: The Mac OS-X kernel (also known as **Darwin**) is also partly based on the Mach microkernel. Another example is QNX, a real-time operating system for embedded systems.

Modules



Perhaps the best current methodology for operating-system design involves using **loadable kernel modules**. Here, the kernel has a set of core components and links in additional services via modules, either at boot time or during run time. This type of design is common in modern implementations of UNIX, such as Solaris, Linux, and Mac OS X, as well as Windows.

The idea of the design is for the kernel to provide core services while other services are implemented dynamically, as the kernel is running. Linking services dynamically is preferable to adding new features directly to the kernel, which would require recompiling the kernel every time a change was made.

Thus, for example, we might build CPU scheduling and memory management algorithms directly into the kernel and then add support for different file systems by way of loadable modules.

The Solaris operating system structure, shown in above Figure, is organized around a core kernel with seven types of loadable kernel modules:

1. Scheduling classes
2. File systems
3. Loadable system calls
4. Executable formats
5. STREAMS modules
6. Miscellaneous
7. Device and bus drivers

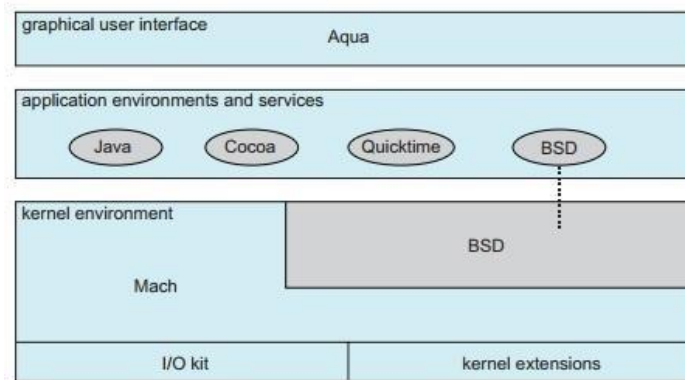
Hybrid Systems

A **hybrid** kernel is an **operating system** kernel architecture that attempts to combine aspects and benefits of microkernel and monolithic kernel architectures used in computer **operating systems**.

In practice, very few operating systems adopt a single, strictly defined structure. Instead, they combine different structures, resulting in hybrid systems that address performance, security, and usability issues. For example, both Linux and Solaris are monolithic, because having the operating system in a single address space provides very efficient performance.

However, they are also modular, so that new functionality can be dynamically added to the kernel. Windows is largely monolithic as well (again primarily for performance reasons), but it retains some behavior typical of microkernel systems, including providing support for separate subsystems (known as operating-system *personalities*) that run as user-mode processes.

Mac OS X



The Mac OS X structure.

The Apple Mac OS X operating system uses a hybrid structure. As shown in Figure , it is a layered system. The top layers include the **Aqua** user interface and a set of application environments and services.

Notably, the **Cocoa** environment specifies an API for the Objective-C programming language, which is used for writing Mac OS X applications. Below these layers is the **kernel environment**, which consists primarily of the Mach microkernel and the BSD UNIX kernel. Mach provides memory management; support for remote procedure calls (RPCs) and interprocess communication (IPC) facilities, including message passing; and thread scheduling.

iOS

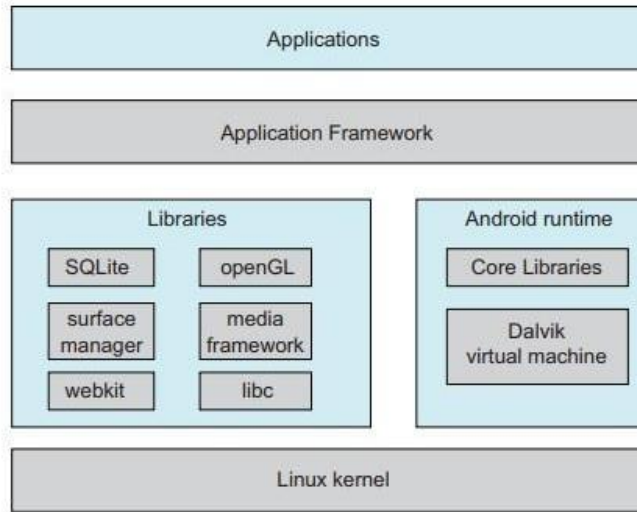


Architecture of Apple's iOS.

iOS is a mobile operating system designed by Apple to run its smartphone, the **iPhone**, as well as its tablet computer, the **iPad**. iOS is structured on the Mac OS X operating system, with added functionality pertinent to mobile devices, but does not directly run Mac OS X applications. The structure of iOS appears in Figure.

Cocoa Touch is an API for Objective-C that provides several frameworks for developing applications that run on iOS devices. The fundamental difference between Cocoa, mentioned earlier, and Cocoa Touch is that the latter provides support for hardware features unique to mobile devices, such as touch screens. The **media services** layer provides services for graphics, audio, and video.

Android



Architecture of Google's Android.

The Android operating system was designed by the Open Handset Alliance (led primarily by Google) and was developed for Android smartphones and tablet computers. Whereas iOS is designed to run on Apple mobile devices and is close-sourced, Android runs on a variety of mobile platforms and is open-sourced, partly explaining its rapid rise in popularity. The structure of

Android appears in Figure 2.18.

Android is similar to iOS in that it is a layered stack of software that provides a rich set of frameworks for developing mobile applications. At the bottom of this software stack is the Linux kernel, although it has been modified by Google and is currently outside the normal distribution of Linux releases.

Software designers for Android devices develop applications in the Java language. However, rather than using the standard Java API, Google has designed a separate Android API for Java development. The Java class files are first compiled to Java bytecode and then translated into an executable file that runs on the Dalvik virtual machine. The Dalvik virtual machine was designed for Android and is optimized for mobile devices with limited memory and CPU processing capabilities.

SYSTEM PROGRAMS

Another aspect of a modern system is its collection of system programs. In computer hierarchy, at the lowest level is hardware. Next is the operating system, then the system programs, and finally the application programs.

System programs, also known as **system utilities**, provide a convenient environment for program development and execution. Some of them are simply user interfaces to system calls. Others are considerably more complex. They can be divided into these categories:

- **File management.** These programs create, delete, copy, rename, print, dump, list, and generally manipulate files and directories.
- **Status information.** Some programs simply ask the system for the date, time, amount of available memory or disk space, number of users, or similar status information. Others are more complex, providing detailed performance, logging, and debugging information.
- **File modification.** Several text editors may be available to create and modify the content of files stored on disk or other storage devices. There may also be special commands to search contents of files or perform transformations of the text.
- **Programming-language support.** Compilers, assemblers, debuggers, and interpreters for common programming languages (such as C, C++, Java, and PERL) are often provided with the operating system or available as a separate download.
- **Program loading and execution.** Once a program is assembled or compiled, it must be loaded into memory to be executed. The system may provide absolute loaders, relocatable loaders, linkage editors, and overlay loaders. Debugging systems for either higher-level languages or machine language are needed as well.
- **Communications.** These programs provide the mechanism for creating virtual connections among processes, users, and computer systems. They allow users to send messages to one another's screens, to browse Web pages, to send e-mail messages, to log in remotely, or to transfer files from one machine to another.
- **Background services.** All general-purpose systems have methods for launching certain system-program processes at boot time. Some of these processes terminate after completing their tasks, while others continue to run until the system is halted. Constantly running system-program processes are known as **services**, **subsystems**, or **daemons**.

One example is process schedulers that start processes according to a specified schedule, system error monitoring services, and print servers. Typical systems have dozens of daemons. In addition, operating systems that run important activities in user context rather than in kernel context may use daemons to run these activities.

Along with system programs, most operating systems are supplied with programs that are useful in solving common problems or performing common operations. Such **application programs** include Web browsers, word processors and text formatters, spreadsheets, database systems, compilers, plotting and statistical-analysis packages, and games.